

SOFTWARE ✨

OVERVIEW

our programming is based on the **Robot Operating System (ROS)**, an environment that provides files and services for communication between programs. This allows us to divide the tasks between several programs at the same time.

CIRCUIT DESIGN

SUB BOARD

- Controls **10** laser distance sensors and a gyroscope via I2C
- Aggregates sensor readings and sends data to the main board through **UART**

MAIN BOARD

Central controller with reserved **PCB interfaces**:

- **2** motor ports (2.54P), driving **4** motors (2 front, 2 rear)
- **3** I2C ports, **4** ADC inputs, and **4** GPIO pins
- **2** UART connections (sub board + camera module)
- Powered by a **12V** battery with a **12V-5V** converter for board operation
- Includes an **OLED** display and three buttons (power, motor enable, start)

HARDWARE ✨

SENSOR & ACTUATOR INTEGRATION

- Path tracking: **9 Laser Sensor VL53LOX** array at front.
- Victim recognition: **Vision modules** (Sipeed MaixCAM) on both sides.
- Terrain perception: **Color sensor** (bottom) for forbidden zones; **bumper switches** (front sides) for collision prevention.
- Drive system: **25GA-370 DC geared motors** with encoders for odometry.

CONTROL & CIRCUIT SYSTEM

- **ESP32** main controller.
- **Two custom PCBs** for power management, signal transmission, and sensor integration, reducing wiring issues and improving **anti-interference**.

CHASSIS & SUSPENSION

- Ground clearance: **25 mm**.
- **Rocker-bogie (parallel linkage) suspension** for terrain adaptability.
- Battery placed at **center-bottom** for **low CG** and **balanced weight distribution**.

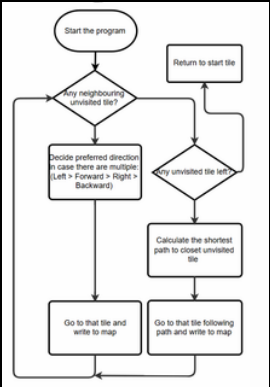
RESCUE KIT DEPLOYMENT SYSTEM

- **Dual-slide track** with a **central rotating disk** for left/right delivery.
- **Bottom guide rail** ensures precise drop into victim zones.

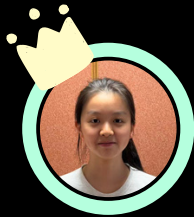
WHEELS & TIRE DESIGN

- **3D-printed resin hubs** with **tooth grooves** for obstacle climbing
- **Ring grooves** lock **silicone tires** firmly, preventing slippage or detachment during turns/climbs.

FLOW-CHART OF THE MAIN LOOP



PCMS_ET



LAO CHI IENG

- Hardware
- Traversing alorithm



LUI IEK TONG

- Hardware
- Documentation



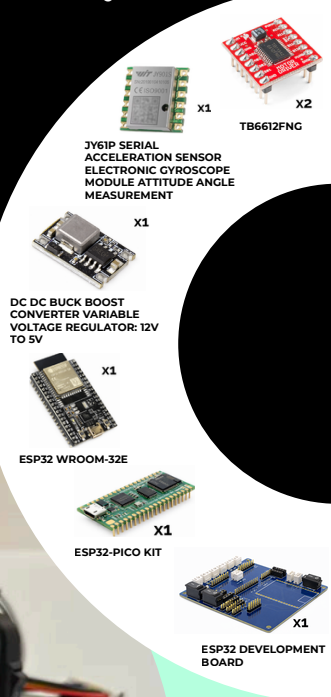
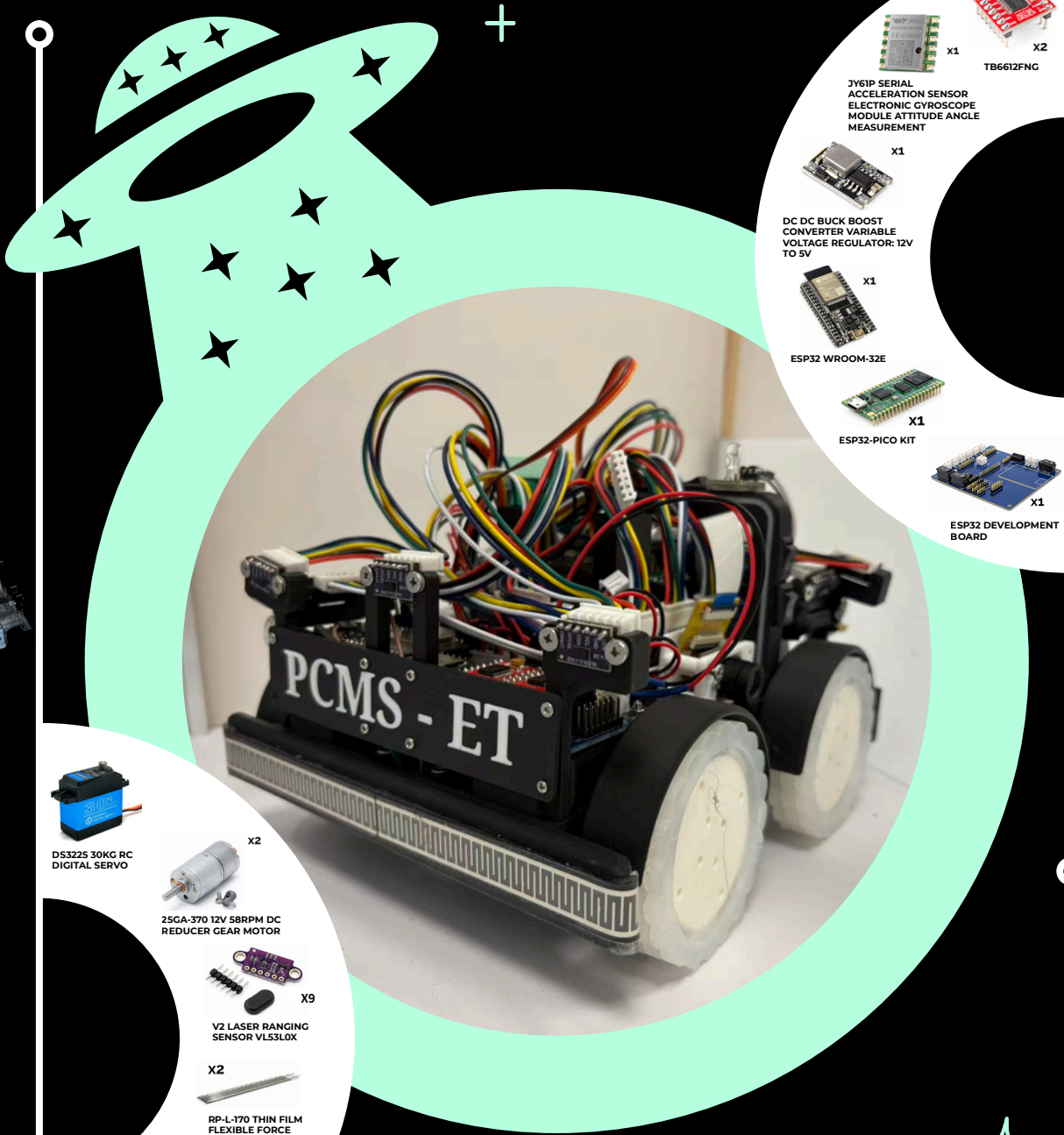
WONG CHI KIO

- Software
- Image recognition



LEI UN HOU

- Logs
- Sensor system



ROBOT & COMPONENT ✨

EXPLORING ALL OF THE MAZE BY SHORTEST ROUTE ✨

Base Method: We designed a maze exploration method that maintains a two-dimensional map of discovered tiles and selects robot actions based on its orientation and local information.

Baseline Reference and Its Limitation: As a reference, we applied a right-hand-based exploration strategy. When the number of maze tiles is N, this approach requires approximately 2.2N search actions on average, resulting in inefficient exploration.

Proposed Solution: To improve efficiency, we developed an original algorithm that combines depth-first search and breadth-first search, named **DBFS (Depth and Breadth First Search)**.

Algorithm Mechanics: DBFS prioritizes movement toward the nearest reachable tile among known candidates. Robot behavior is divided into search and move, defined below.

SEARCH

Adjacent unexplored tiles are added to a **LIFO** structure named **UNSEARCH** with the priority:
[LEFT → FRONT → RIGHT]

MOVE

The robot selects the next destination from **UNSEARCH**. Using **BFS**, the robot computes the shortest path and runs a brief simulation before moving.



RESULT

DBFS was evaluated against a right-hand-based method. The number of search actions was reduced by approximately **33%**, improving exploration efficiency and enabling more reliable exit reach and **Exit Bonus** achievement

IMAGE RECOGNITION USING MACHINE LEARNING ✨

Victims on maze walls are detected while the robot moves forward using **UART communication** with camera modules. Color victims are identified using **LAB thresholding**, while letter victims are recognized by a trained machine learning model. White LEDs are used to reduce the influence of ambient lighting, improving recognition accuracy.



Machine Learning Details: Letter recognition uses a **MobileNet-V2** model trained on approximately **6,000 self-collected images**. The model is converted to **TensorFlow Lite** and deployed for real-time inference

Training progress is monitored using learning curves to prevent overfitting, and recognition accuracy improves with continued training.